



intrinsics

 [CV] Chessboard 관련 함수들: OpenCV → 참고

 카메라 내부 파라미터(intrinsics) 구할 때 실행하는 코드 설명

 `python ../scripts/compute_intrinsics.py --folder_images ./frames/ -ich 6 -icw 8 -s 30 -t 24 --debug`

→ 실행 방법

```
python compute_intrinsics.py --folder_images ./frames_cam0 -ich 6 -icw 8 -s 30 -t 24 --debug
```

• 실행할 때 작성하는 명령어

`python compute_intrinsics.py`는 해당 파이썬 파일을 실행해라

그 뒷부분은 다 문자열로 인식됨

• parser 부분에 대한 내용은 추후에 정리에정

→ 필요한 규칙들 제외하고는 빼면 됨 / 뺄 때 main 함수에 있는 내용도 같이 빼야 오류 안남 !!!!

```
main(**vars(args))
```

```
1 main(folder_images = "./frames_cam0",
2       inner_corners_height = 6,
3       inner_corners_width = 8,
4       square_sizes = 30,
5       threads = 24,
6       debug = True,
7       )
```

→ `def(main)`이 실행됨

```
1 (cv2_major, cv2_minor, _) = cv2.__version__.split(".")
2 if int(cv2_major)<4:
```

```
3 raise ImportError("Opencv version 4+ required!")
```

- OpenCV 주버전 4이상으로 강제

def 관련 정의 내용

def process_image

```
1 def process_image(filename_image, inner_corners_height,
2   inner_corners_width, debug, debug_folder):
3     print("Processing image {} ...".format(filename_image))
4
5     gray = utils.rgb2gray(imageio.imread(filename_image))
6     #체커보드 검출은 흑백에서 더 잘되는 편이라 rgb2gray 하는거임
7
8     # Find the chess board corners
9     # ret, corners = cv2.findChessboardCorners(gray,
10    (inner_corners_height, inner_corners_width),
11    #
12    cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE)
13    # findChessboardCorners -> findChessboardCornersSB로 수정함
14    # OpenCV 버전 바뀌면서 더 성능 좋아진거라고 하던데 ~?
15    ret, corners = cv2.findChessboardCornersSB(gray,
16    (inner_corners_height, inner_corners_width),
17    cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE)
18
19    # If found, add object points, image points (after refining them)
20    if not ret:
21        return None
22
23    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 0.001)
24    imgp = cv2.cornerSubPix(gray, corners, (11,11), (-1,-1), criteria)
25
26    if debug:
27        gray = cv2.drawChessboardCorners(gray,
28    (inner_corners_height, inner_corners_width), imgp, ret)
29        imageio.imwrite(os.path.join(debug_folder,
30    os.path.basename(filename_image)), gray)
31
32    return np.float32(imgp)
```

- 이미지 1장을 읽어 체커보드 코너를 찾고(못 찾으면 패스), 서브픽셀로 더 정확히 다듬어서 좌표를 돌려주는 함수

▽ 코너 찾기 1단계 (대략 좌표 잡기)

코너 찾기 1단계(대략 좌표 잡기)

- 패턴 크기 (H, W) 로 내부 코너 개수를 정확히 넣어야 함.
- 플래그:
 - ADAPTIVE_THRESH : 밝기 들쭉날쭉해도 견고하게
 - NORMALIZE_IMAGE : 명암 보정해서 안정화
- 결과:
 - ret : 찾았는지(True/False)
 - corners : 대략 코너 위치들 (H*W, 1, 2)

cv2.cornerSubPix (2차 정밀 좌표 잡기)

cornerSubPix: 이미 찾은 코너의 **대략 좌표**(픽셀 단위)를, 주변 작은 창을 보면서 **서브픽셀(픽셀보다 더 촘촘한)** 정밀도로 **여러 번** 미세조정해 주는 함수.

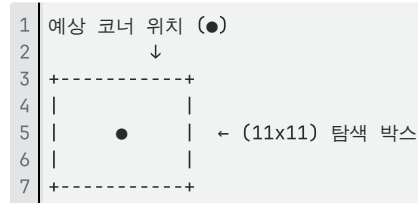
🔴 탐색 윈도우란?

- 이미 코너가 대충 **(x, y)** 위치에 있다고 알고 있어.
(findChessboardCorners가 알려줌)
- 근데 그게 딱 맞진 않고, 약간 옆에 있을 수도 있음.
- 그래서 코너 주변을 ****작은 네모 박스(윈도우)****로 잘라내서, 그 안에서만 “진짜 코너”를 다시 찾는 거야.

(11, 11) 의미

- **(11, 11)** 이면 **가로 11픽셀 × 세로 11픽셀**짜리 네모 박스야.
- 즉, “대략적인 코너 좌표 중심에서, 왼쪽/오른쪽/위/아래로 5픽셀씩” 둘러보면서 최적의 코너 위치를 찾는 거지.

그림으로 표현하면:



바꿔볼 만한 요소 01

findChessboardCorners vs **findChessboardCornersSB**

```
. ret, corners = cv2.findChessboardCorners( image, patternSize, flags=None )
```

조명 반사나 시야 왜곡등이 존재하면 → **findChessboardCornersSB** ** OpenCV에서도 이걸 권장한다고 함

findChessboardCorners → 근데 기존 코드로 실행했을 때는 애 성능이 더 좋게 나오긴함 // 이걸 바꿔가면서 해 봐야함

- SB는 다른 플래그를 씁니다 (**EXHAUSTIVE** , **ACCURACY** 등). 위 두 옵션은 SB에선 의미 없거나 무시될 수 있음

```
1 flags = 0  
2 if hasattr(cv2, "CALIB_CB_EXHAUSTIVE"): flags |=  
  cv2.CALIB_CB_EXHAUSTIVE # 더 집요하게 탐색(느림, 성공률↑)  
3 if hasattr(cv2, "CALIB_CB_ACCURACY"): flags |= cv2.CALIB_CB_ACCURACY  
  # 정확도↑(약간 느림)  
4 ret, corners = cv2.findChessboardCornersSB(gray, (H, W), flags=flags)
```

바꿔볼 만한 요소 02

```
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,  
0.001)
```

1. MAX_ITER = 30

→ “최대 30번만 위치를 더 다듬자.”

2. EPS = 0.001

→ “한 번 더 다듬었을 때 이동량이 0.001픽셀보다 작으면(거의 안 움직이면) 이제 충분히 맞췄다!”

빨리 하고 싶다 → (10, 0.01) : 빠름(정밀도↓)

더 정밀하게 하고 싶다 → (50, 0.0005) : 느리지만 더 촘촘